

Ex3_1_RandomProcessExperiments-sol2

Prof M.Tognolini HEIG-VD

March 4, 2026

1 Processus Aléatoires (Ex 3.1)

M.Tognolini - 02 mars 2026 HEIG-VD rev 2.0

2 1. Configuration de l'environnement

```
[13]: import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# Configuration pour des graphiques clairs
%matplotlib inline
plt.rcParams['figure.figsize'] = [10, 6]
```

2.1 2. Génération du Processus Aléatoire

Un processus aléatoire $X(n, \zeta)$ est une famille de fonctions du temps. Nous générons ici N réalisations de longueur L . Un exemple classique de cet exercice est un processus sinusoïdal avec une phase aléatoire ϕ uniformément distribuée sur $[0, 2\pi]$.

$$X(n) = A \cos(\omega_0 n + \phi)$$

L'exercice porte généralement sur la génération de multiples réalisations (chemins) d'un processus et l'analyse de ses propriétés statistiques (moyenne d'ensemble vs moyenne temporelle). Nous utilisons des fonctions de génération de nombres aléatoires pour créer ces réalisations et les stocker dans une matrice. Chaque colonne de la matrice représente une réalisation du processus, tandis que chaque ligne correspond à un instant de temps. La phase de la fonction cos est également randomisée pour chaque réalisation afin de simuler un processus aléatoire.

Pour générer un processus sinusoïdal avec une phase aléatoire, nous pouvons utiliser la fonction `numpy.random.uniform` pour générer des phases aléatoires et `numpy.cos` pour créer les réalisations du processus. Voici un exemple de code pour générer un tel processus :

Pour avoir toujours la même séquence aléatoire à chaque exécution, nous pouvons fixer la graine du générateur de nombres aléatoires avec `numpy.random.seed`. Cela garantit que les résultats sont reproductibles.

```
import numpy as np
import matplotlib.pyplot as plt
# Paramètres du processus
```

```

A = 1.0          # Amplitude
f0 = 0.02        # Fréquence normalisée (cycles par échantillon)
N = 100          # Nombre d'échantillons par réalisation
NR = 50          # Nombre de réalisations
# Fixer la graine pour la reproductibilité
np.random.seed(0)
# Générer des phases aléatoires uniformément distribuées entre 0 et 2π
phi = np.random.uniform(0, 2 * np.pi, NR)
# Générer les réalisations du processus
n = np.arange(N) # Vecteur de temps
X = A * np.cos(2 * np.pi * f0 * n[:, None] + phi) # Matrice de réalisations (N x NR)
# Afficher quelques réalisations
plt.figure(figsize=(12, 6))
for i in range(5): # Afficher les 5 premières réalisations
    plt.plot(n, X[:, i], label=f'Réalisation {i+1}')
plt.title('Réalisation d'un processus sinusoïdal avec phase aléatoire')
plt.xlabel('Temps (échantillons)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid()
plt.show()

```

Ce code génère un processus sinusoïdal avec une phase aléatoire et affiche les premières réalisations. Chaque réalisation est une fonction du temps avec une phase différente, illustrant ainsi la nature aléatoire du processus.

```

[14]: # 1. Fixer la graine (seed) pour la reproductibilité
      # N'importe quel entier fonctionne, mais 42 est souvent utilisé par convention.
      np.random.seed(0)

      # Paramètres
      NR = 100 # Nombre de réalisations (lignes) [cite: 76]
      N = 1000 # Nombre d'échantillons (colonnes) [cite: 73]
      A = 1.0
      F0 = 1/50 # Fréquence normalisée (cycles par échantillon)
      omega0 = 2 * np.pi * F0
      n = np.arange(N)
      Anoise = 0.0

      # 2. Génération de la phase (une par réalisation)
      # On utilise NR pour la taille afin d'avoir une phase par ligne [cite: 79-80]
      phi = np.random.uniform(0, 2 * np.pi, size=(NR, 1))

      # 3. Création de la matrice des processus (NR x N)
      # Chaque ligne est une réalisation, chaque colonne est un échantillon [cite: 80]
      X = A * np.cos(omega0 * n + phi)

```

```

# 4. Ajout du bruit blanc gaussien
# Même taille que la matrice X (NR x N)

noise = Anoise * np.random.normal(0, 0.1, size=(NR, N))
X = X + noise

print(f"Forme de la matrice X : {X.shape}") # Doit être (100, 1000)
print("Les 5 premières valeurs de la réalisation 1 :")
print(X[0, :5])

# 1. Définition de l'indice des réalisations (de 0 à NR-1)
nr = np.arange(NR)

# 2. Extraction du 3ème échantillon pour toutes les lignes (réalisations)
# X[:, 2] sélectionne toutes les lignes et la colonne d'index 2 (le 3ème
    ↳ échantillon)
sample_3 = X[:, 2]

# Affichage des premières réalisations
plt.figure(figsize=(12, 4))
plt.plot(nr[:100], X[:5, :100].T)
plt.title("5 premières réalisations du processus $X(n)$")
plt.xlabel("Temps (n)")
plt.ylabel("$X(n)$")
plt.grid(True)
plt.show()

# 3. Affichage graphique (équivalent au subplot(2,1,2) de votre code MATLAB)
plt.figure(figsize=(10, 4))
plt.plot(nr, sample_3, label='x(3)')

# Configuration des axes
plt.title('Valeur du 3ème échantillon à travers les 100 réalisations')
plt.xlabel('Numéro de la réalisation (nr)')
plt.ylabel('Valeur de 1\'échantillon X(3)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)

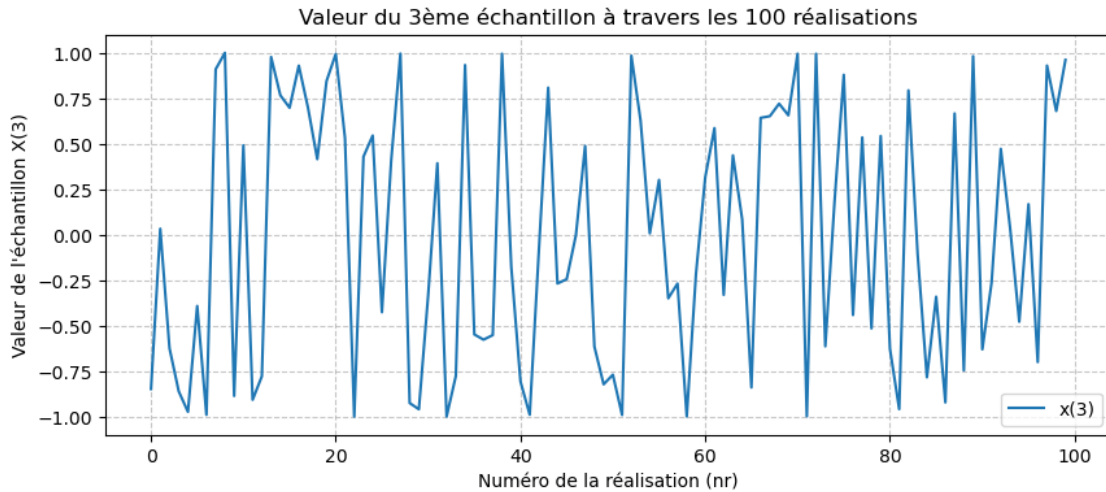
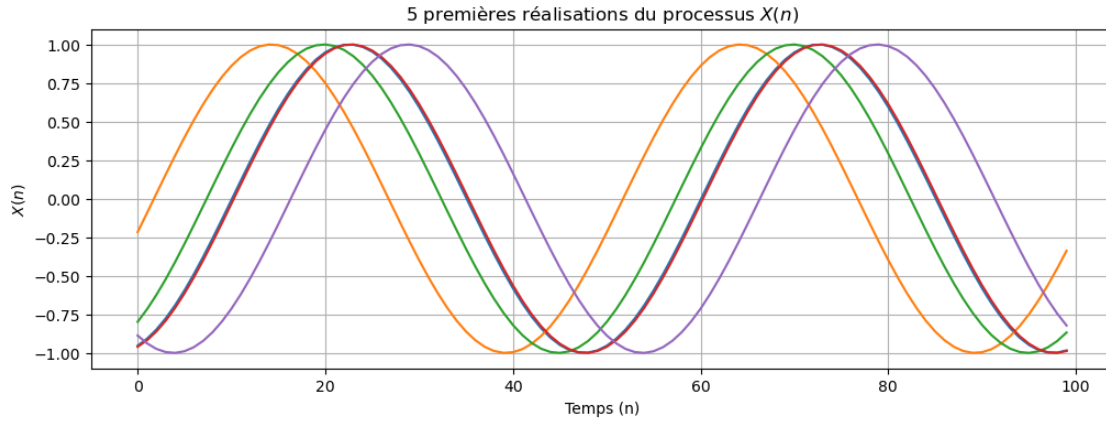
plt.show()

```

Forme de la matrice X : (100, 1000)

Les 5 premières valeurs de la réalisation 1 :

[-0.95333378 -0.90797605 -0.848299 -0.77524377 -0.68996248]



2.2 3. Statistiques d'Ensemble

La moyenne d'ensemble (Ensemble Mean) est calculée en faisant la moyenne de toutes les réalisations à un instant n donné :

$$\mu_x(n) = E[X(n)] = (1/N) \sum_{i=1}^N X_i(n)$$

où $X_i(n)$ est la i -ème réalisation du processus à l'instant n . La moyenne d'ensemble est un vecteur de taille n (nombre d'échantillons temporels).

```
[15]: # 1. Moyenne d'ensemble (Ensemble Average, chaque colonne de X représente une
      ↪ réalisation)
      # La moyenne d'ensemble est calculée en prenant la moyenne de chaque ligne de la
      ↪ matrice X, ce qui correspond à la moyenne de toutes les réalisations à chaque
      ↪ instant de temps.
```

```

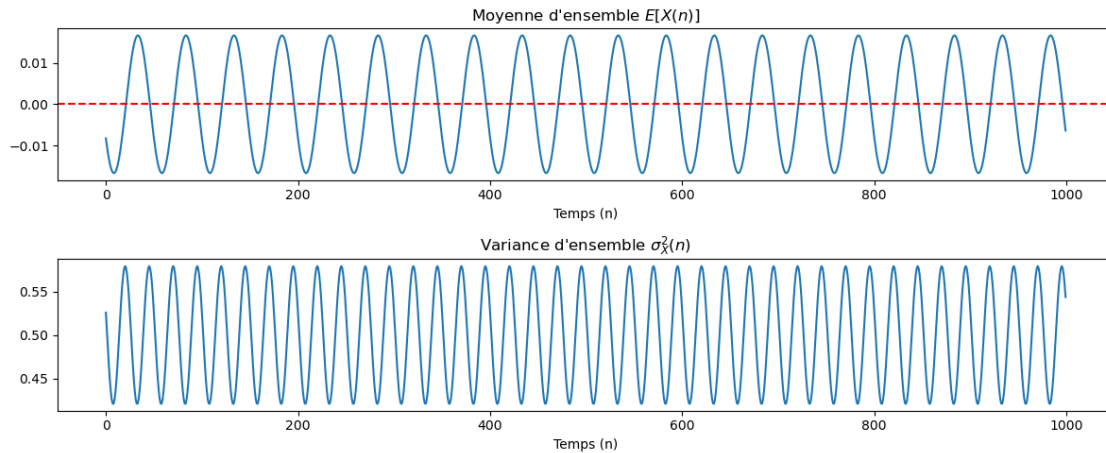
# La moyenne d'ensemble est un vecteur de taille n (nombre d'échantillons ↪ temporels).
# Moyenne d'ensemble
ensemble_mean = np.mean(X, axis=0)

# Variance d'ensemble
ensemble_var = np.var(X, axis=0)

plt.figure(figsize=(12, 5))
plt.subplot(2, 1, 1)
plt.plot(n, ensemble_mean)
plt.xlabel("Temps (n)")
plt.axhline(y=0, color='r', linestyle='--')
plt.title("Moyenne d'ensemble  $E[X(n)]$ ")

plt.subplot(2, 1, 2)
plt.plot(n, ensemble_var)
plt.title("Variance d'ensemble  $\sigma_X^2(n)$ ")
plt.xlabel("Temps (n)")
plt.tight_layout()
plt.show()

```



2.2.1 Consatations :

La moyenne d'ensemble d'un processus sinusoïdal avec une phase aléatoire uniforme est nulle, car les différentes réalisations s'annulent mutuellement en moyenne.

La varance d'ensemble (Ensemble Variance) est calculée en utilisant la formule :

$$\sigma_x^2(n) = E[(X(n) - \mu_x(n))^2] = (1/N) \sum_{i=1}^N (X_i(n) - \mu_x(n))^2$$

où $\mu_x(n)$ est la moyenne d'ensemble calculée précédemment. La variance d'ensemble est également un vecteur de taille n.

La variance d'ensemble d'un processus sinusoïdal avec une phase aléatoire uniforme est égale à $A^2/2$, car la puissance du signal est répartie uniformément entre les différentes réalisations.

2.3 4. Moment de second ordre: Autocorrélation et Ergodicité

L'autocorrélation d'ensemble $R_{XX}(n1, n2)$ mesure la dépendance entre deux instants. Pour un processus stationnaire, elle ne dépend que de l'écart $k = n1 - n2$.

$$R_{XX}(k) = E[X(n)X(n+k)]$$

Pour un processus sinusoïdal avec une phase aléatoire uniforme, l'autocorrélation d'ensemble est donnée par :

$$R_{XX}(k) = \frac{A^2}{2} \cos(2\pi f_0 k)$$

Cela montre que l'autocorrélation d'ensemble d'un processus sinusoïdal avec une phase aléatoire uniforme est une fonction cosinus de l'écart k , avec une amplitude égale à $A^2/2$.

$R_{XX}(n1, n2)$ mesure la dépendance entre deux instants. Pour un processus stationnaire, elle ne dépend que de l'écart $k = n1 - n2$.

L'ergodicité est une propriété qui permet de relier les statistiques d'ensemble aux statistiques temporelles. Un processus est ergodique si la moyenne temporelle d'une réalisation est égale à la moyenne d'ensemble.

```
# X est la matrice (NR x N) générée précédemment
# Calcul de la matrice d'autocorrélation Rx (N x N)
# En Python: (1/NR) * X.T @ X
Rx = (1 / NR) * (X.T @ X) # Calcul de la fonction d'autocorrélation d'ensemble Rxx(k)
# Rx est une matrice de taille N x N, où chaque élément Rx[i, j] correspond à
# l'autocorrélation entre les échantillons i et j.
# Pour un processus stationnaire, on peut extraire la fonction
# d'autocorrélation en fonction du lag k = i - j.
# La fonction d'autocorrélation d'ensemble Rxx(k) est obtenue
# en prenant la moyenne des éléments de Rx pour chaque lag k

print(f"Taille de la matrice Rx : {Rx.shape}") # Doit être (1000, 1000)
print(f"Exemple de valeurs dans Rx :\n {Rx[:5, :5]}") # Affiche les 5 premiers éléments de la
# première ligne pour vérification
# 1. Calcul du vecteur Rxx par moyennage des diagonales
# La matrice Rx est de taille N x N
# Les lags vont de -(N-1) à +(N-1)
lags = np.arange(-(N - 1), N)
rxx_ensemble = np.zeros(len(lags))
print(f"Diagonale pour k=0 : {np.diag(Rx, k=0)[:5]}") # Affiche les 10 premiers éléments
# de la diagonale principale pour vérification
print(f"Diagonale pour k=1 : {np.diag(Rx, k=1)[:5]}")
print(f"Valeur moyenne de la diagonale k=1 : {np.mean(np.diag(Rx, k=1))}") # Affiche la moyenne
# de la diagonale principale pour vérification
for i, k in enumerate(lags):
    # np.diag(Rx, k) extrait la k-ième diagonale
```

```

    # k=0 est la diagonale principale, k>0 au-dessus, k<0 au-dessous
    diagonal = np.diag(Rx, k)
    rxx_ensemble[i] = np.mean(diagonal)
# 2. Visualisation du résultat
plt.figure(figsize=(10, 5))
plt.plot(lags, rxx_ensemble, label='Rxx(k) moyenné sur les diagonales')
plt.title('Fonction d\'autocorrélation d\'ensemble $R_{xx}(k)$')
plt.xlabel('Lag $k$')
plt.ylabel('Amplitude')
plt.grid(True)
plt.xlim([-100, 100]) # Zoom pour voir la structure
plt.legend()

```

[25]:

```

# X est la matrice (NR x N) générée précédemment
# Calcul de la matrice d'autocorrélation Rx (N x N)
# En Python: (1/NR) * X.T @ X
Rx = (1 / NR) * (X.T @ X) # Calcul de la fonction d'autocorrélation d'ensemble
    ↳ Rxx(k)
# Rx est une matrice de taille N x N, où chaque élément Rx[i, j] correspond à
    ↳ l'autocorrélation entre les échantillons i et j.
# Pour un processus stationnaire, on peut extraire la fonction d'autocorrélation
    ↳ en fonction du lag k = i - j.
# La fonction d'autocorrélation d'ensemble Rxx(k) est obtenue en prenant la
    ↳ moyenne des éléments de Rx pour chaque lag k

print(f"Taille de la matrice Rx : {Rx.shape}") # Doit être (1000, 1000)
print(f"Exemple de valeurs dans Rx :\n {Rx[:5, :5]}") # Affiche les 5 premiers
    ↳ éléments de la première ligne pour vérification

# on peut comparer les valeurs de rxx_esensemble avec la première ligne de Rx
    ↳ pour les lags positifs et la première colonne pour les lags négatifs
rx_etime_Rx_pos = Rx[0, 0:-1] # La première ligne de Rx correspond à lags
    ↳ positifs
rx_etime_Rx_neg = Rx[:, 0] # La première colonne de Rx correspond à lags
    ↳ négatifs
rx_estimate = np.concatenate((rx_etime_Rx_neg[::-1], rx_etime_Rx_pos)) # On
    ↳ concatène les lags négatifs (inversés) et positifs
print(f"r_x_estimate shape : {rx_estimate.shape}") # Doit être (1999,) pour les
    ↳ lags de -999 à +999

# 1. Calcul du vecteur Rxx par moyennage des diagonales
# La matrice Rx est de taille N x N
# Les lags vont de -(N-1) à +(N-1)
lags = np.arange(-(N - 1), N)
rxx_ensemble = np.zeros(len(lags))
print(f"Diagonale pour k=0 : {np.diag(Rx, k=0)[:5]}") # Affiche les 10 premiers
    ↳ éléments de la diagonale principale pour vérification

```

```

print(f"Diagonale pour k=1 : {np.diag(Rx, k=1)[:5]}")
print(f"Valeur moyenne de la diagonale k=1 : {np.mean(np.diag(Rx, k=1))}") #
    ↳ Affiche la moyenne de la diagonale principale pour vérification
for i, k in enumerate(lags):
    # np.diag(Rx, k) extrait la k-ième diagonale
    # k=0 est la diagonale principale, k>0 au-dessus, k<0 au-dessous
    diagonal = np.diag(Rx, k)
    rxx_ensemble[i] = np.mean(diagonal)
# 2. Visualisation du résultat
plt.figure(figsize=(10, 5))
plt.plot(lags, rxx_ensemble, 'b.-', label='Rxx(k) moyenné sur les diagonales')
plt.plot(lags, rx_estimate, marker='+', label='Rxx(k) estimé à partir de Rx[0,:
    ↳ ],', linestyle='--')
plt.title('Fonction d\'autocorrélation d\'ensemble $R_{xx}(k)$')
plt.xlabel('Lag $k$')
plt.ylabel('Amplitude')
plt.grid(True)
plt.xlim([-100, 100]) # Zoom pour voir la structure
plt.axhline(y=0.5, color='r', linestyle='--') # Ligne horizontale à y=0.5 pour
    ↳ référence
plt.axhline(y=0.0, color='g', linestyle='--') # Ligne horizontale à y=0.0 pour
    ↳ référence
plt.axhline(y=-0.5, color='k', linestyle='--') # Ligne horizontale à y=-0.5 pour
    ↳ référence

plt.legend()

```

Taille de la matrice Rx : (1000, 1000)

Exemple de valeurs dans Rx :

```

[[0.52586697 0.51231586 0.49068522 0.46131619 0.42467192]
 [0.51231586 0.50639364 0.4924853  0.47081016 0.44171007]
 [0.49068522 0.4924853  0.48651858 0.47287918 0.45178219]
 [0.46131619 0.47081016 0.47287918 0.46749061 0.45472943]
 [0.42467192 0.44171007 0.45178219 0.45472943 0.45050532]]

```

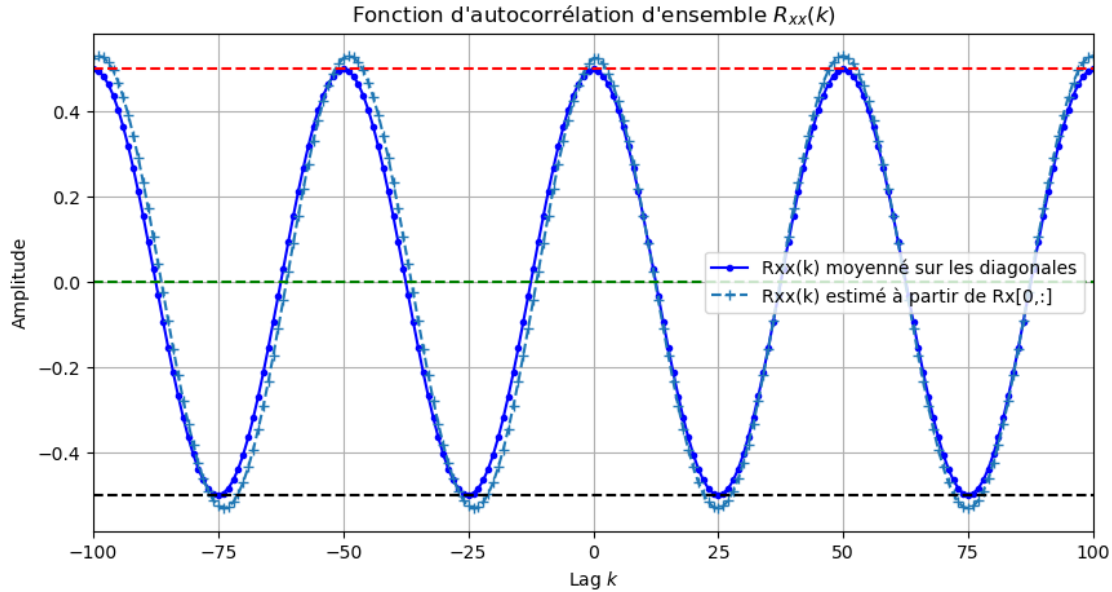
r_x_estimate shape : (1999,)

Diagonale pour k=0 : [0.52586697 0.50639364 0.48651858 0.46749061 0.45050532]

Diagonale pour k=1 : [0.51231586 0.4924853 0.47287918 0.45472943 0.43917647]

Valeur moyenne de la diagonale k=1 : 0.4960222480670715

[25]: <matplotlib.legend.Legend at 0x1658d1950>



2.3.1 2. Définition de l'autocorrélation théorique

Pour une sinusoïde à phase aléatoire, la fonction d'autocorrélation théorique dépend uniquement de l'écart $k-l$:

$$r_{xx}(k, l) = \frac{A^2}{2} \cos(2\pi f_0(k - l))$$

pour comparer avec la première ligne de R_x (où $k=0$), on utilise:

```
# k représente ici le décalage (lag)
k_lags = np.arange(N)
# r_x théorique pour k=0 : rx = 0.5 * A^2 * cos(n * w0)
# Calcul de la fonction d'autocorrélation théorique
rx_theorique = 0.5 * (A**2) * np.cos(k_lags * omega0)
```

```
[32]: # k représente ici le décalage (lag)
k_lags = np.arange(N)
# r_x théorique pour k=0 : rx = 0.5 * A^2 * cos(n * w0)
rx_theorique = 0.5 * (A**2) * np.cos(k_lags * omega0) #
print(f"forme de rx_theorique : {rx_theorique.shape}") # Doit être (1999,)
# Extraction de la première ligne de la matrice estimée
# rx_estim_ligne1 = Rx[0, :] #
rx_estim = rxx_ensemble[N-1:N-1+N] # Rx[0, :] # Rx[-(N-1) , N] #

print(f"forme de rx_estim : {rx_estim.shape}") # Doit être (1000,)

# Calcul de l'erreur d'estimation
erreur = rx_theorique - rx_estim #
Eerr = np.sum(erreur**2) # Erreur quadratique totale
```

```

print(f"Erreur quadratique totale : {Eerr}")

plt.figure(figsize=(12, 10))

# Tracé de la comparaison
plt.subplot(2, 1, 1)
plt.plot(k_lags, rx_estim, 'x-', label='Estimée  $r_{xx}$  ensemble$', markevery=1)
    ↪ #
plt.plot(k_lags, rx_theorique, 'r-', label='Théorique  $r_x(k=0, 1)$ ') #
plt.plot(k_lags[0:N-1], rx_etime_Rx_pos[0:N], marker='+', label='Rxx(k) estimé à
    ↪ partir de Rx[0,:]', linestyle='--' ) #
plt.plot(k_lags, erreur, 'y', label='Erreur d\'estimation de Estimée  $r_{xx}$ 
    ↪ ensemble$') #

# Configuration selon le document MATLAB
plt.title('Comparaison Autocorrélation : Théorique vs Numérique ( $k=0$ )')
plt.xlabel('Décalage (1)')
plt.ylabel(' $r_x(k-1)$ ')
plt.axis([0, 50, -0.6, 0.6]) # Zoom sur les 50 premiers lags pour la visibilité
plt.legend()
plt.grid(True)

plt.subplot(2, 1, 2)

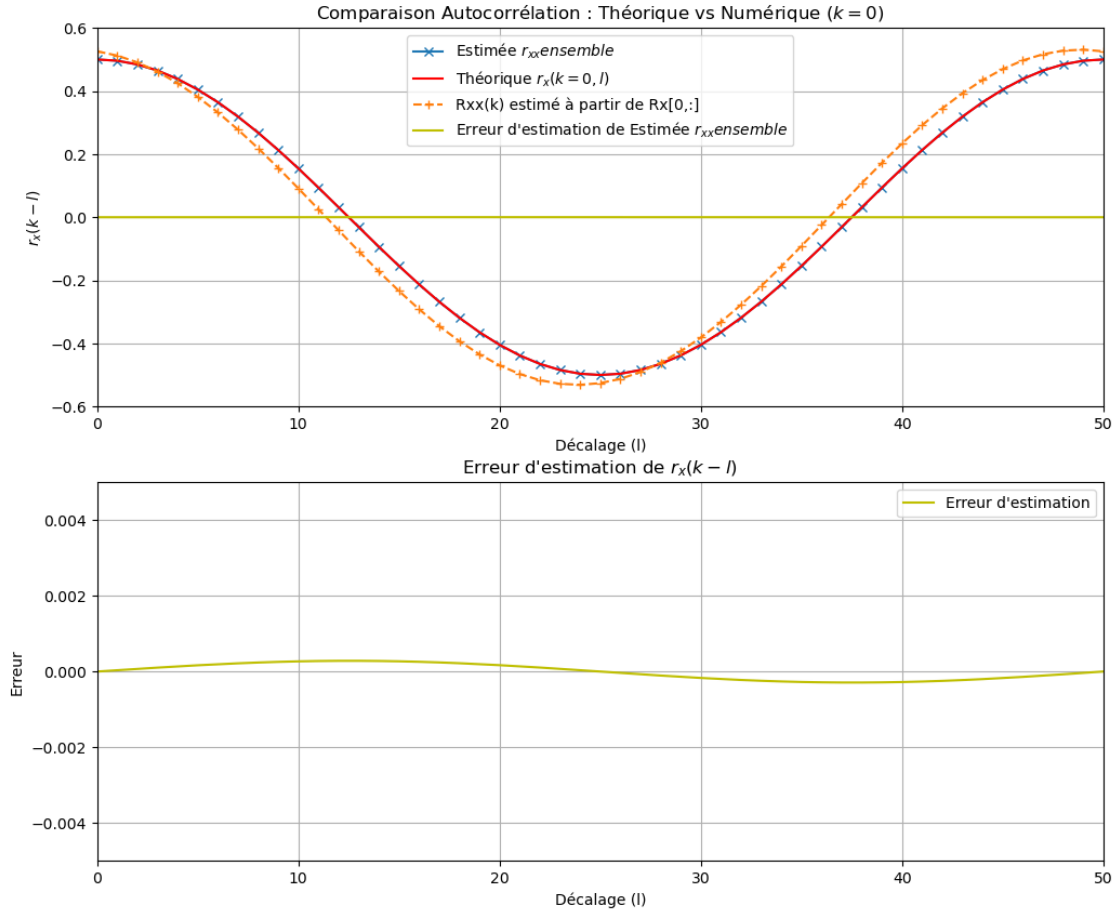
plt.plot(k_lags, erreur, 'y', label='Erreur d\'estimation')
plt.title('Erreur d\'estimation de  $r_x(k-1)$ ')
plt.xlabel('Décalage (1)')
plt.ylabel('Erreur')
plt.axis([0, 50, -0.005, 0.005]) # Zoom sur les 50 premiers lags pour la
    ↪ visibilité
plt.legend()
plt.grid(True)
plt.show()

```

```

forme de rx_theorique : (1000,)
forme de rx_estim : (1000,)
Erreur quadratique totale : 0.014795525443526589

```



2.3.2 Analyse des résultats:

- **Symétrie** : La matrice R_x obtenue est symétrique car elle résulte du produit $X^T X$.
- **Convergence** : Pour un grand nombre de réalisations ($NR=100$), l'estimation numérique R_x converge vers la fonction théorique r_x .
- **Erreur** : L'erreur d'estimation (en jaune) montre les fluctuations résiduelles dues au nombre fini de réalisations.

2.4 Power spectrum Density (PSD)

La densité spectrale de puissance (DSP) est obtenue en appliquant la transformée de Fourier à la fonction d'autocorrélation. On va appliquer la transformée de Fourier à la fonction d'autocorrélation d'ensemble $R_{xx}(k)$ pour obtenir la DSP du processus. La DSP nous donne une représentation de la distribution de la puissance du signal en fonction de la fréquence.

```
# 1. Calcul de la DSP via la Transformée de Fourier (FFT) des autocorrélations
# On utilise fftshift pour centrer la fréquence zéro
rxx_estim = rxx_ensemble[N - N//2 : N + N//2]
```

```

# on prends que les lags de  $-N/2$  à  $N/2$  pour centrer la fonction
# d'autocorrélation autour de zéro
print(f"Forme de rxx_estim : {rxx_estim.shape}") # Doit être (1000,)
Sx = 1/(N) * np.abs(np.fft.fftshift(np.fft.fft(rxx_estim)))
# Rx[0, :] est la première ligne de la matrice
# d'autocorrélation, qui correspond à  $r_{xx}(k)$  pour  $k=0$ 
#print(f"Forme de rxx_ensemble : {rxx_ensemble.shape}") # Doit être (1000,)
print(f"Forme de Sx : {Sx.shape}") # Doit être (1000,)
Var_x = np.sum(Sx) # La variance du signal x
# peut être obtenue en intégrant la DSP sur toutes les fréquences, ce qui correspond
# à la somme de Sx multipliée par  $N/2$  (car Sx est normalisée par N)
print(f"Variance du signal x calculée à partir de la DSP : {Var_x}")
# Il faut le comparer avec la DSP théorique d'
# une sinusoïde à phase aléatoire, qui est un pic à la fréquence  $f_0$  :

# pour la DSP theorique on prend rx_theorique avec
# un lag de  $-N/2$  à  $N/2$  pour centrer la fonction
# d'autocorrélation autour de zéro
rx_theorique_red = rx_theorique[N - N//2: N + N//2]
# on prends que les lags de  $-N/2$  à  $N/2$  pour centrer la fonction
# d'autocorrélation autour de zéro
print(f"Forme de rx_theorique_red : {rx_theorique_red.shape}") # Doit être (1000,)
Sx_theorique = 1/(N) * (np.abs(np.fft.fftshift(np.fft.fft(rx_theorique_red))))
print(f"Forme de Sx_theorique : {Sx_theorique.shape}") # Doit être (1000,)
# La variance du signal x peut être obtenue en intégrant la DSP sur toutes les fréquences,
# ce qui correspond à la somme de Sx_theorique multipliée par  $N/2$ 
# (car Sx_theorique est normalisée par N)
Var_x_theorique = np.sum(Sx_theorique)

print(f"Variance du signal x calculée à partir de la DSP théorique : {Var_x_theorique}")
# 2. Conversion des DSP en décibels (dB)
Sx_dB = 10 * np.log10(Sx + 1e-12) # Ajout d'une constante infime pour éviter le log(0)
Sx_theorique_dB = 10 * np.log10(Sx_theorique + 1e-12) # Ajout d'une constante infime
# pour éviter le log(0)
# 3. Visualisation de la DSP
# Fréquences associées à la DSP
frequencies = np.fft.fftshift(np.fft.fftfreq(len(rxx_estim), d=1))

```

[33]:

```

# 1. Calcul de la DSP via la Transformée de Fourier (FFT) des autocorrélations
# On utilise fftshift pour centrer la fréquence zéro
rxx_estim = rxx_ensemble[N - N//2: N + N//2] # on prends que les lags de  $-N/2$ 
→ à  $N/2$  pour centrer la fonction d'autocorrélation autour de zéro
print(f"Forme de rxx_estim : {rxx_estim.shape}") # Doit être (1000,)

```

```

Sx = 1/(N) * np.abs(np.fft.fftshift(np.fft.fft(rxx_estim))) # Rx[0, :] est la
↳ première ligne de la matrice d'autocorrélation, qui correspond à rxx(k) pour
↳ k=0
#print(f"Forme de rxx_ensemble : {rxx_ensemble.shape}") # Doit être (1000,)
print(f"Forme de Sx : {Sx.shape}") # Doit être (1000,)
Var_x = np.sum(Sx) # La variance du signal x peut être obtenue en intégrant la
↳ DSP sur toutes les fréquences, ce qui correspond à la somme de Sx multipliée
↳ par N/2 (car Sx est normalisée par N)
print(f"Variance du signal x calculée à partir de la DSP : {Var_x}")
# Il faut le comparer avec la DSP théorique d'une sinusoïde à phase aléatoire,
↳ qui est un pic à la fréquence f0 :
# pour la DSP theorique on prend rx_theorique avec un lag de -N/2 a N/2 pour
↳ centrer la fonction d'autocorrélation autour de zéro
rx_theorique_red = rx_theorique[N - N//2: N + N//2] # on prends que les lags de
↳ -N//2 à N//2 pour centrer la fonction d'autocorrélation autour de zéro
print(f"Forme de rx_theorique_red : {rx_theorique_red.shape}") # Doit être
↳ (1000,)
Sx_theorique = 1/(N) * (np.abs(np.fft.fftshift(np.fft.fft(rx_theorique))))
print(f"Forme de Sx_theorique : {Sx_theorique.shape}") # Doit être (1000,)
Var_x_theorique = np.sum(Sx_theorique) # La variance du signal x peut être
↳ obtenue en intégrant la DSP sur toutes les fréquences, ce qui correspond à la
↳ somme de Sx_theorique multipliée par N/2 (car Sx_theorique est normalisée par
↳ N)
print(f"Variance du signal x calculée à partir de la DSP théorique :
↳ {Var_x_theorique}")
# 2. Conversion des DSP en décibels (dB)
Sx_dB = 10 * np.log10(Sx + 1e-12) # Ajout d'une constante infime pour éviter le
↳ log(0)
Sx_theorique_dB = 10 * np.log10(Sx_theorique + 1e-12) # Ajout d'une constante
↳ infime pour éviter le log(0)
# 3. Visualisation de la DSP
frequencies = np.fft.fftshift(np.fft.fftfreq(len(rxx_estim), d=1)) # Fréquences
↳ associées à la DSP
print(f"Forme de frequencies : {frequencies.shape}") # Doit être (1000,)
plt.figure(figsize=(10, 5))
plt.subplot(2, 1, 1)
plt.plot(frequencies, Sx, 'b+-', label='DSP (linéaire)')
plt.plot(frequencies, Sx_theorique, 'r-', label='DSP théorique (linéaire)')
plt.title('Densité Spectrale de Puissance (DSP) du processus')
plt.ylabel('DSP (linéaire)')
plt.grid(True)
plt.xlim([-0.05, 0.05]) # Limiter l'axe des fréquences
plt.legend()
plt.subplot(2, 1, 2)
plt.plot(frequencies, Sx_dB, 'b+-', label='DSP (dB)')
plt.plot(frequencies, Sx_theorique_dB, 'r-', label='DSP théorique (dB)')

```

```
#plt.plot(frequencies, Sx_theorique_dB, label='DSP théorique (dB)')

plt.xlabel('Fréquence normalisée (cycles par échantillon)')
plt.ylabel('DSP (dB)')
plt.grid(True)
plt.xlim([-0.05, 0.05]) # Limiter l'axe des fréquences
plt.legend()
plt.show()
```

Forme de rxx_estim : (1000,)

Forme de Sx : (1000,)

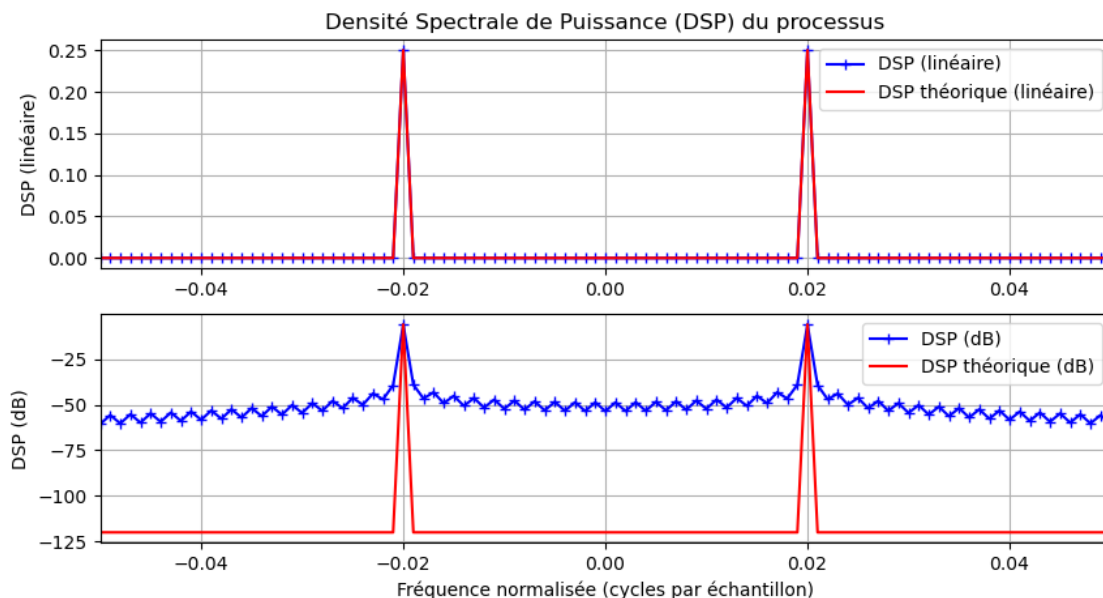
Variance du signal x calculée à partir de la DSP : 0.5015840388847477

Forme de rx_theorique_red : (500,)

Forme de Sx_theorique : (1000,)

Variance du signal x calculée à partir de la DSP théorique : 0.50000000000000226

Forme de frequencies : (1000,)



```
[19]: # Valeur propres de la matrice d'autocorrélation Rx
eigenvalues, eigenvectors = np.linalg.eig(Rx)
LambdaMax = np.max(eigenvalues/N)
LambdaMin = np.min(eigenvalues/N)
print(f"Valeur max des valeurs propres de Rx normalisées par N : {LambdaMax} ")
print(f"Valeur min des valeurs propres de Rx normalisées par N : {LambdaMin} ")
```

Valeur max des valeurs propres de Rx normalisées par N : (0.2896846440019632+0j)

Valeur min des valeurs propres de Rx normalisées par N :

(-6.930861935505296e-17+0j)

2.5 Autocorrelation of a sum of process (Exemple 3.3.3)

Consider the exemple in slide 45. $y(n)$ is a sum of 2 random processes: A sum of M sinusoids with a random phase like the process $x(n)$ above and the white gaussian noise $v(n)$.

$$y(n) = x(n) + v(n) = \left[\sum_{m=1}^M A_m \sin(n\omega_m + \phi_m) \right] + v(n)$$

In the exemple $A_m = A$ and $\omega_m = \omega_0$ are constant with m , and ϕ_m a random variable uniformly distributed like in the previous exemple. The noise $v(n)$ is a white gaussian noise with a variance σ^2 .

First create $y(n)$ process using the matrix previously defined xn and use the function `randn` to generate the white gaussian noise. With $M = 5$ and $w_m = w_0/m$ and $A_m = A_0/m$ with ϕ_m a uniform distributed phase like in ex 3.1.1, compute the sum of the harmonic process $yh(n)$ and the gaussian noise $v(n)$ produced with the function `randn()` with a std of A_0 .

```
[34]: # Fixer la graine pour la reproductibilité
np.random.seed(0)

# Paramètres de base
NR = 100      # Nombre de réalisations
N = 1000     # Nombre d'échantillons
AO = 5.0     # Amplitude de référence
w0 = 2*1/50 * np.pi
M = 5        # Nombre de sinusoides
n = np.arange(N)

# Initialisation du processus harmonique yh(n)
yh = np.zeros((NR, N))

# Construction de la somme de sinusoides (yh)
for m in range(1, M + 1):
    Am = AO / m
    wm = w0 * m
    print(f"Sinusoïde {m}: Amplitude = {Am:.2f}, Fréquence angulaire = {wm:.4f} rad/s")
    # Génération d'une phase aléatoire différente pour chaque sinusoides et
    # chaque réalisation
    phi_m = np.random.uniform(0, 2 * np.pi, size=(NR, 1))
    #print(f"Phase aléatoire pour sinusoides {m} (exemple de 5 réalisations) : \n{phi_m[:5, 0]}") # Affiche les phases aléatoires pour les 5 premières
    #réalisations
    print(f"Phase aléatoire pour sinusoides {m} (exemple de 3 réalisations) : \n{phi_m[:3, 0]}")
    # Ajout de la composante harmonique
    yh += Am * np.sin(wm * n + phi_m)
print(f"Forme de yh : {yh.shape}") # Doit être (100, 1000)
```

```

# Génération du bruit blanc gaussien v(n)
# L'écart-type est égal à A0
v = np.random.normal(0, A0, size=(NR, N))
print(f"Forme de v : {v.shape}") # Doit être (100, 1000)
print(f"Puissance du signal harmonique yh : {np.mean(yh**2):.2f}")
print(f"Puissance du bruit v : {np.mean(v**2):.2f}")

# Somme finale
y = yh + v

```

Sinusoïde 1: Amplitude = 5.00, Fréquence angulaire = 0.1257 rad/s
Phase aléatoire pour sinusoïde 1 (exemple de 3 réalisations) :
[3.44829694 4.49366732 3.78727399]
Sinusoïde 2: Amplitude = 2.50, Fréquence angulaire = 0.2513 rad/s
Phase aléatoire pour sinusoïde 2 (exemple de 3 réalisations) :
[4.2588469 1.69651013 4.61936028]
Sinusoïde 3: Amplitude = 1.67, Fréquence angulaire = 0.3770 rad/s
Phase aléatoire pour sinusoïde 3 (exemple de 3 réalisations) :
[1.9590713 4.37525518 2.37348481]
Sinusoïde 4: Amplitude = 1.25, Fréquence angulaire = 0.5027 rad/s
Phase aléatoire pour sinusoïde 4 (exemple de 3 réalisations) :
[5.69605619 4.86348283 2.09321272]
Sinusoïde 5: Amplitude = 1.00, Fréquence angulaire = 0.6283 rad/s
Phase aléatoire pour sinusoïde 5 (exemple de 3 réalisations) :
[2.5211878 5.83891018 0.62589907]
Forme de yh : (100, 1000)
Forme de v : (100, 1000)
Puissance du signal harmonique yh : 18.30
Puissance du bruit v : 24.87

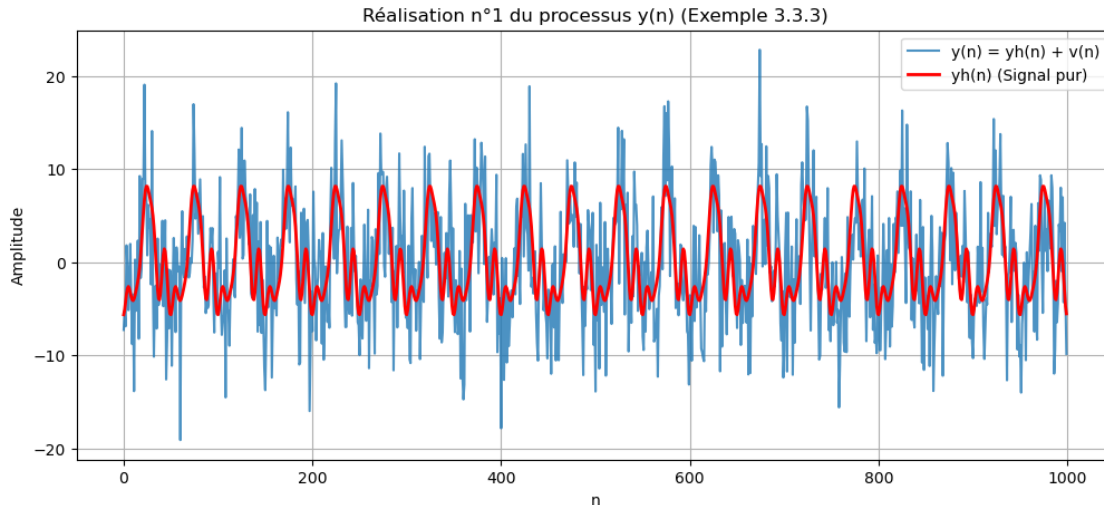
2.5.1 Visualisation de une réalisation de $y(n)$:

Il est utile d'afficher une réalisation pour voir l'aspect du signal (mélange de sinusoïdes lentes et de bruit fort).

```

[35]: plt.figure(figsize=(12, 5))
plt.plot(n, y[1, :], label='y(n) = yh(n) + v(n)', alpha=0.8)
plt.plot(n, yh[1, :], 'r', label='yh(n) (Signal pur)', linewidth=2)
plt.title('Réalisation n°1 du processus y(n) (Exemple 3.3.3)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True)
plt.show()

```

2.5.2 Points clés à retenir:

- **Les fréquences ω_m** : Avec $\omega_m = \omega_0 * m$, plus m augmente, plus la sinusoïde est rapide (haute fréquence).
- **Les amplitudes A_m** : Avec $A_m = A_0/m$, les amplitudes décroissent avec m , ce qui signifie que les composantes de plus haute fréquence sont plus faibles que les composantes de basse fréquence.
- **Le bruit $v(n)$** : Avec un écart-type de A_0 , le bruit est assez dominant, car il a la même puissance que la sinusoïde la plus forte.
- **Indépendance** : Chaque phase ϕ_m est générée indépendamment pour chaque harmonique, ce qui est crucial pour les propriétés d'autocorrélation.

2.5.3 Calcul des autocorrélations d'ensemble et temporelle de $y(n)$:

Pour démontrer que l'autocorrélation d'une somme de processus indépendants est égale à la somme de leurs autocorrélations, nous allons procéder par estimation statistique sur nos $NR=100$ réalisations.

Comme vous l'avez souligné, la condition théorique est l'absence de corrélation entre le signal harmonique $y_h(n)$ et le bruit $v(n)$.

Pour le calcul de la corrélation estimé sur toutes les réalisations on utilise la fonction `compute_avg_autocorr` définie ci-dessus.

```
def compute_avg_autocorr(matrix):
    """Calcule la corrélation moyenne sur toutes les réalisations (lignes)."""
    n_real, n_samp = matrix.shape
    # Taille du résultat : 2*N - 1
    corr_sum = np.zeros(2 * n_samp - 1)
```

```

for i in range(n_real):
    # On utilise le mode 'full' pour avoir les lags de -(N-1) à +(N-1)
    # On divise par N pour obtenir l'estimation biaisée standard (Puissance à k=0)
    corr_sum += signal.correlate(matrix[i, :], matrix[i, :], mode='full') / n_samp

return corr_sum / n_real

```

si on veut une estimation non biaisée, on peut diviser par $(N - |k|)$ au lieu de N pour chaque lag k , mais cela complique un peu le code. L'estimation biaisée est plus simple et souvent suffisante pour les démonstrations théoriques.

```

def compute_avg_autocorr_unbiased(matrix):
    n_real, n_samp = matrix.shape
    lags = np.arange(-n_samp + 1, n_samp)

    # Initialisation de la somme des corrélations
    corr_sum = np.zeros(len(lags))

    # Facteur de normalisation sans biais :  $N - |k|$ 
    # On évite la division par zéro si jamais on va jusqu'à  $N$  (ici max  $N-1$ )
    normalization = n_samp - np.abs(lags)

    for i in range(n_real):
        # Calcul de la corrélation brute (somme directe)
        raw_corr = signal.correlate(matrix[i, :], matrix[i, :], mode='full')
        # Normalisation sans biais pour chaque lag  $k$ 
        corr_sum += raw_corr / normalization

    # Moyenne sur toutes les réalisations
    return corr_sum / n_real

```

```

[39]: def compute_avg_autocorr(matrix):
    """Calcule la corrélation moyenne sur toutes les réalisations (lignes)."""
    n_real, n_samp = matrix.shape
    # Taille du résultat :  $2*N - 1$ 
    corr_sum = np.zeros(2 * n_samp - 1)

    for i in range(n_real):
        # On utilise le mode 'full' pour avoir les lags de -(N-1) à +(N-1)
        # On divise par N pour obtenir l'estimation biaisée standard (Puissance
        →à k=0)
        corr_sum += signal.correlate(matrix[i, :], matrix[i, :], mode='full') /
        →n_samp

    return corr_sum / n_real
def compute_avg_autocorr_unbiased(matrix):
    n_real, n_samp = matrix.shape
    lags = np.arange(-n_samp + 1, n_samp)

```

```

# Initialisation de la somme des corrélations
corr_sum = np.zeros(len(lags))

# Facteur de normalisation sans biais :  $N - |k|$ 
# On évite la division par zéro si jamais on va jusqu'à  $N$  (ici max  $N-1$ )
normalization = n_samp - np.abs(lags)

for i in range(n_real):
    # Calcul de la corrélation brute (somme directe)
    raw_corr = signal.correlate(matrix[i, :], matrix[i, :], mode='full')
    # Normalisation sans biais pour chaque lag  $k$ 
    corr_sum += raw_corr / normalization

# Moyenne sur toutes les réalisations
return corr_sum / n_real

# 1. Calcul des autocorrélations moyennes
ryh = compute_avg_autocorr_unbiased(yh)
rv = compute_avg_autocorr_unbiased(v)
ry = compute_avg_autocorr_unbiased(y)
print(f"Forme de ryh : {ryh.shape}") # Doit être (1999,)
print(f"Forme de rv : {rv.shape}") # Doit être (1999,)
print(f"Forme de ry : {ry.shape}") # Doit être (1999,)

# 2. Vecteur des lags (de  $-N+1$  à  $N-1$ )
lags = np.arange(-N + 1, N)

# 3. Extraction des valeurs à  $k = 0$  (le centre du vecteur)
idx_zero = len(lags) // 2
ryh_0 = ryh[idx_zero]
rv_0 = rv[idx_zero]
ry_0 = ry[idx_zero]

# 4. Affichage du graphique
plt.figure(figsize=(12, 7))

plt.plot(lags, ry, 'b-+', label=f'$r_y(k)$ (Somme), $r_y(0)={ry_0:.3f}$',
         alpha=0.6)
plt.plot(lags, ryh, 'r', label=f'$r_{{yh}}(k)$ (Harmonique), $r_{{yh}}(0)={ryh_0:.3f}$')
plt.plot(lags, rv, 'g', label=f'$r_v(k)$ (Bruit), $r_v(0)={rv_0:.3f}$')

plt.title('Vérification de l\'additivité des autocorrélations')
plt.xlabel('Lag (k)')
plt.ylabel('Autocorrélation')
plt.legend()
plt.grid(True)

```

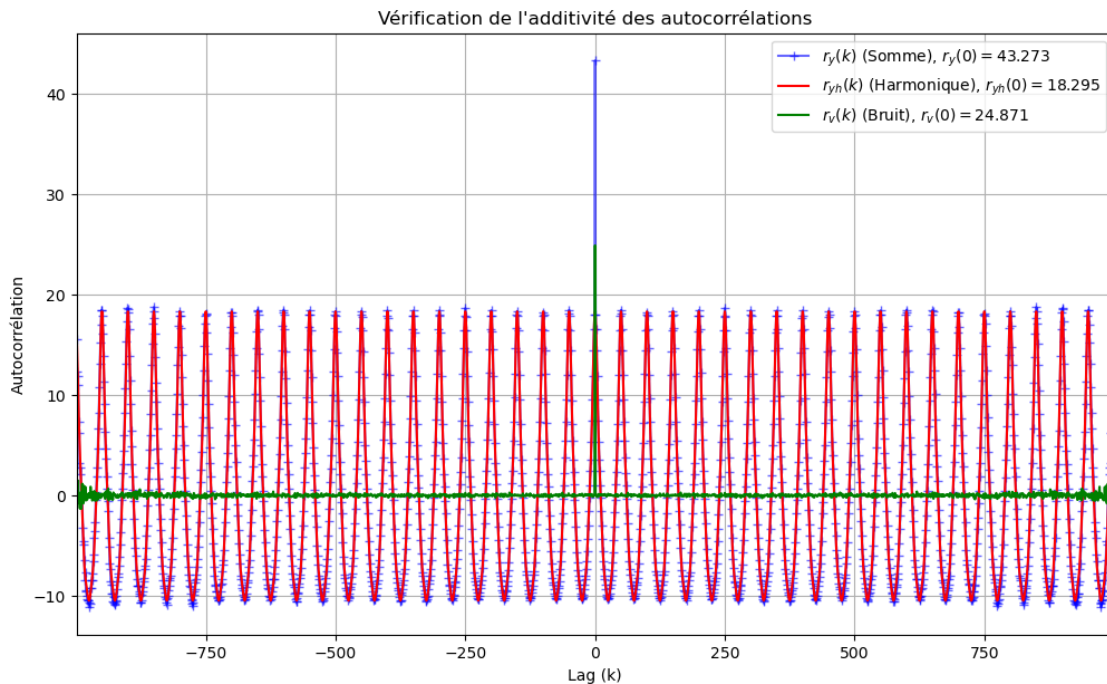
```
plt.xlim([-999, 999]) # Zoom pour voir la structure du bruit à k=0
plt.show()

# 5. Affichage des résultats numériques pour vérification
print(f"--- Valeurs à k = 0 ---")
print(f"ryh(0)           : {ryh_0:.4f}")
print(f"rv(0)              : {rv_0:.4f}")
print(f"Somme (théorie)    : {ryh_0 + rv_0:.4f}")
print(f"ry(0) (mesuré)     : {ry_0:.4f}")
print(f"Différence         : {abs(ry_0 - (ryh_0 + rv_0)):.4e}")
```

Forme de ryh : (1999,)

Forme de rv : (1999,)

Forme de ry : (1999,)



```
--- Valeurs à k = 0 ---
ryh(0)           : 18.2951
rv(0)            : 24.8709
Somme (théorie)  : 43.1661
ry(0) (mesuré)   : 43.2732
Différence       : 1.0711e-01
```

2.5.4 Analyse des résultats

- **Comportement de r_v** : La fonction d'autocorrélation du bruit blanc gaussien $r_v(k)$ est caractéristique. Elle est maximale à $k=0$, où elle prend la valeur de la variance du bruit

$\sigma^2 = A_v^2 = 25$. Pour tous les autres lags ($k \neq 0$), $r_v(k)$ est proche de zéro, indiquant l'absence de corrélation entre les échantillons du bruit à différents instants.

- **Comportement de r_{yh}** : La fonction d'autocorrélation du signal harmonique $r_{yh}(k)$ présente une structure périodique due à la nature sinusoïdale du signal. Elle est maximale à $k=0$, où elle prend la valeur de la puissance totale du signal harmonique, qui est la somme des puissances de chaque harmonique. Les autres lags montrent des oscillations qui reflètent les différentes fréquences présentes dans le signal. Sa valeur à $k=0$ est la puissance totale du signal harmonique :

$$r_{yh}(0) = \sum_{m=1}^M \frac{A_m^2}{2} = \frac{A_0^2}{2} \sum_{m=1}^M \frac{1}{m^2} \approx 18.2951...$$

- **Comportement de r_y** : La fonction d'autocorrélation du signal total $r_y(k)$ est la somme de $r_{yh}(k)$ et $r_v(k)$, car les deux processus sont indépendants. Ainsi, à $k=0$, $r_y(0) = r_{yh}(0) + r_v(0) \approx 18.2951 + 25 = 43.2951$. Pour les autres lags, $r_y(k)$ présente une structure périodique similaire à celle de $r_{yh}(k)$, mais avec une amplitude plus élevée à $k=0$ en raison de la contribution du bruit. La très légère différence provient du fait que nous utilisons un nombre fini de réalisations ($NR=100$), mais l'erreur tend vers zéro quand $NR \rightarrow \infty$.

2.6 Calcul de la densité spectrale de puissance DSP

La densité spectrale de puissance (DSP) d'un processus aléatoire peut être estimée à partir de sa fonction d'autocorrélation en utilisant la transformée de Fourier (en utilisant le théorème de Wiener-Khinchin).

Passer à une échelle logarithmique en décibels (dB) est une pratique standard en traitement du signal. Cela permet de visualiser simultanément des pics de forte puissance (les sinusoïdes) et des niveaux de bruit beaucoup plus faibles qui seraient invisibles sur une échelle linéaire.

La formule utilisée est : $S_{dB} = 10 \log_{10}(S)$

```
[46]: # 1. Calcul de la DSP via la Transformée de Fourier (FFT) des autocorrélations
# On utilise fftshift pour centrer la fréquence zéro
Syh = 1/N * np.abs(np.fft.fftshift(np.fft.fft(ryh)))
print(f"Forme de Syh : {Syh.shape}") # Doit être (1999,)
Sv = 1/N * np.abs(np.fft.fftshift(np.fft.fft(rv)))
Sy = 1/N * np.abs(np.fft.fftshift(np.fft.fft(ry)))
# Conversion des DSP en décibels (dB)
# On ajoute une constante infime (1e-12) pour éviter le log(0)
Syh_dB = 10 * np.log10(Syh + 1e-12)
Sv_dB = 10 * np.log10(Sv + 1e-12)
Sy_dB = 10 * np.log10(Sy + 1e-12)
# 2. Définition de l'axe des fréquences normalisées (de -0.5 à 0.5)
freqs = np.linspace(-0.5, 0.5, len(ryh))

# 3. Affichage des résultats en dB
plt.figure(figsize=(12, 8))
```

```

# Sous-graphique 1 : Composantes individuelles en dB
plt.subplot(2, 1, 1)
plt.plot(freqs, Syh_dB, 'r', label='DSP de $y_h$ (Harmonique)')
plt.plot(freqs, Sy_dB, 'b', label='DSP de $y$ (Somme mesurée)', alpha=0.7 )
plt.plot(freqs, Sv_dB, 'g', label='DSP de $v$ (Bruit)')
plt.title('Densité Spectrale de Puissance (DSP) en dB - Composantes_
↳individuelles')
plt.ylabel('Puissance (dB)')
plt.ylim([-40, 30]) # Ajustement de l'échelle pour voir le plancher de bruit
plt.xlim([-0.11, 0.11]) # Limites de fréquence normalisée
plt.legend()
plt.grid(True, which='both', linestyle='--', alpha=0.5)

# Sous-graphique 2 : Somme des processus en dB
plt.subplot(2, 1, 2)
plt.plot(freqs, Sy_dB, 'b', label='DSP de $y$ (Somme mesurée)')
# Note : Pour la somme théorique en dB, on additionne d'abord les puissances_
↳linéaires
Sy_sum_theo_dB = 10 * np.log10(Syh + Sv + 1e-12)
plt.plot(freqs, Sy_sum_theo_dB, 'r-.', label='Somme théorique_
↳($10\log_{10}(S_{yh} + S_v)$)', alpha=0.7)

plt.title('Vérification de l\'additivité en échelle logarithmique (dB)')
plt.xlabel('Fréquence normalisée ($f/f_s$)')
plt.ylabel('Puissance (dB)')
plt.ylim([-40, 30])
plt.xlim([-0.11, 0.11]) # Limites de fréquence normalisée
plt.legend()
plt.grid(True, which='both', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

#5. Affichage des spectres de puissance en échelle linéaire pour une meilleure_
↳visualisation de l'additivité
plt.figure(figsize=(12, 8))
plt.plot(freqs, Syh, 'r', label='DSP de $y_h$ (Harmonique)')
plt.plot(freqs, Sy, 'b', label='DSP de $y$ (Somme mesurée)', alpha=0.7 )
plt.plot(freqs, Sv, 'g', label='DSP de $v$ (Bruit)')
plt.title('Densité Spectrale de Puissance (DSP) en échelle linéaire -_
↳Composantes individuelles')
plt.ylabel('Puissance')
plt.xlim([-0.11, 0.11]) # Limites de fréquence normalisée
plt.legend()
plt.grid(True, which='both', linestyle='--', alpha=0.5)
plt.show()

```

Forme de Syh : (1999,)

